App Dev 1 Project Report

Trekking Management Application (TMA)

1. Student Details

Name: Prerak Manojkumar Shah

Roll Number: 23f3001390

Email: 23f3001390@ds.study.iitm.ac.in

About Me: I am pursuing a degree in Mechanical Engineering at COEP (College of Engineering Pune), alongside the IIT Madras BS Degree program, where I've developed a strong interest in full-stack web application development. Working on this project helped me get hands-on with design thinking and system architecture, going from a rough idea to a structured, role-based application with clear data flows and user experience considerations. I enjoy building practical systems that combine role-based workflows, data management, and clear user experience.

2. Project Details

Project Title: Trekking Management Application (TMA)

Problem Statement:

To design and build a web-based application that allows administrators, trek staff, and trekkers to manage trekking expeditions end-to-end — including trek creation and lifecycle management, staff onboarding and approval, slot-based bookings, participant tracking, and role-specific dashboards with analytics.

Approach:

The app was built using Flask as the backend framework with a modular Blueprint structure (auth, admin, staff, user). SQLAlchemy models persist data in SQLite. Role-based access via Flask-Login separates Admin, Staff, and Trekker workflows. Jinja2 templates with Bootstrap 5 provide a responsive UI, and Chart.js visualizes booking and trek statistics on dashboards.

3. AI/LLM Declaration

I used Cursor (AI coding assistant) and Claude (Anthropic) throughout this project, particularly for areas where I had limited prior experience. Since I wasn't very fluent with HTML, CSS, and Bootstrap, I relied on AI assistance to write and structure frontend markup and styling, and to understand how various Bootstrap classes worked. I also used Cursor for debugging — several bugs in the application (including logic and query errors that weren't immediately obvious) were identified with AI assistance, which helped me get the app to a stable, bug-free state. Additionally, AI was used to draft API documentation (api.yaml), structure the project report, README.md and improve consistency in documentation wording.

The extent of AI/LLM usage is around 35–40%, covering frontend styling/markup assistance, debugging support, and documentation. All core application logic, database design, route implementation, and feature integration decisions were done by me, with AI used as a support tool rather than a replacement for understanding the code.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy / SQLAlchemy	ORM for SQLite database models and queries
Jinja2	Template engine for rendering dynamic HTML pages

Technology / Library	Purpose
Bootstrap 5	Frontend styling and responsive layout
Bootstrap Icons	UI icons across dashboards and navigation
Chart.js	Visualize trek, booking, and activity trends
Flask-Login	User authentication and session management
Werkzeug	Secure password hashing and verification
SQLite	Lightweight local database for storing application data

5. Database Schema / ER Diagram

Tables:

- users — stores account details: id (PK), name, email (unique), contact_number, whatsapp_number, password, role (admin / staff / trekker), status (active / blacklisted), created_at, updated_at
- treks — stores trek expeditions: id (PK), name, location, difficulty (Easy / Moderate / Hard), duration (days), total_slots, available_slots, assigned_staff_id (FK → users), status (Pending / Approved / Open / Closed / Completed), start_date, end_date, created_at, updated_at
- bookings — links trekkers to treks: id (PK), user_id (FK → users), trek_id (FK → treks), booking_date, booking_status (Booked / Cancelled / Completed), payment_status (Pending / Paid), created_at, updated_at
- staff_profiles — staff specialization and approval: id (PK), user_id (FK → users, 1:1), specialization, experience (years), status, approved_by_admin, created_at, updated_at

Relationships:

- One-to-Many → User → Booking
- One-to-Many → Trek → Booking
- One-to-One → User → StaffProfile
- Many-to-One → Trek.assigned_staff_id → User (staff)

Trek lifecycle (admin, forward-only):

Pending → Approved → Open → Closed / Completed (Closed can also move to Completed)



Figure 1: ER Diagram — generated with dbdiagram.io

6. API Resource Endpoints

The application uses form-based Flask routes with session authentication (Flask-Login). Primary resources:

Endpoint	Method	Description
`/login`	POST	Authenticate user and create session
`/register/trekker`	POST	Register a new trekker account
`/register/staff`	POST	Register staff (pending admin approval)
`/logout`	GET	End user session
`/admin/dashboard`	GET	Admin stats and overview charts
`/admin/trekk/create`	POST	Create a new trek
`/admin/treks`	GET	List and manage all treks
`/admin/trek/<id>/edit`	GET/POST	Edit trek details
`/admin/trek/<id>/delete-confirm`	GET/POST	Delete a trek
`/admin/trek/<id>/status/<status>`	POST	Advance trek lifecycle status
`/admin/staff/pending`	GET	List staff pending approval

Endpoint	Method	Description
/admin/staff/approve/<user_id>	POST	Approve pending staff
/admin/staff/reject/<user_id>	POST	Reject pending staff
/admin/staff/all	GET	View all staff
/admin/users	GET	Manage trekker accounts
/admin/user/<id>/toggle-status	POST	Toggle trekker active/blacklisted
/admin/bookings	GET	View all bookings
/admin/booking/<id>/cancel	POST	Cancel a booking (admin)
/staff/dashboard	GET	Staff dashboard for assigned treks
/staff/trek/<id>	GET	View assigned trek + participants
/staff/trek/<id>/update-slots	POST	Update available slots
/staff/trek/<id>/update-status	POST	Update assigned trek status
/staff/trek/<id>/remove-participant/<booking_id>	POST	Remove a participant
/user/dashboard	GET	Trekker analytics dashboard
/user/treks	GET	List open treks (optional ?search=)
/user/trek/<id>	GET	Trek details before booking
/user/book-trek/<trek_id>	POST	Book a seat on an open trek
/user/booking-confirmation/<id>	GET	Booking confirmation page
/user/booking-history	GET	Trekker booking history
/user/cancel-booking/<id>	POST	Cancel an active booking

YAML API Definition File: Included separately in the submission ZIP as api.yaml.

7. Architecture and Features

Architecture Overview:

- run.py — Application entry point; starts Flask dev server
- config.py — App config (SECRET_KEY, SQLite URI sqlite:///tma.db)
- app/__init__.py — Flask app factory, DB/login init, blueprint registration, default admin bootstrap
- app/models.py — SQLAlchemy models: User, Trek, Booking, StaffProfile
- app/auth.py — Login, logout, trekker/staff registration
- app/admin_routes.py — Admin dashboards, trek CRUD, staff/user/booking management
- app/staff_routes.py — Assigned-trek operations (slots, status, participants)
- app/user_routes.py — Trek listing, booking, cancellation, history, charts
- app/templates/ — Jinja2 HTML templates by role (admin/, staff/, user/)
- app/static/ — CSS and front-end assets
- app/seeds.py — Optional sample data seeding

Implemented Features:

- Role-based registration and login (Admin, Staff, Trekker)

- Staff onboarding with admin approval workflow
- Trek creation, editing, deletion, and forward-only lifecycle status updates
- Slot-based booking and cancellation for trekkers
- Staff tools to update slots/status and remove participants
- Admin management of users (active/blacklist), staff, treks, and bookings
- Visualization dashboards using Chart.js (overview, history, trends)
- Responsive UI built with Bootstrap 5
- Secure password hashing (Werkzeug) and session handling (Flask-Login)

Additional Features:

- Search/filter for open treks by name
- Booking confirmation page after successful reservation
- Optional database seed script for demo data
- Trek status stepper UI for clearer lifecycle visibility

8. Video Presentation

Drive Link: https://drive.google.com/file/d/1hY9NWLJfdjJ6vlP406tIoODNe_t743A9/view?usp=sharing